MODULE 13 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

RAG in an application

Chunking, embeddings, retrieval, grounded prompts, citations, security trimming and troubleshooting, end to end.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 13 of 39

admin@sthithapragna.com

RAG quality is decided **before the model speaks**: what you chunk, how you embed, what you retrieve, and what you tell the model to do with it.

admin@sthithapragna.com

From documents to a grounded answer

01
Chunk
Split documents into focused passages.

02
Embed
Turn each chunk into a vector.

03
Index
Store text, vectors and citation fields.

04
Retrieve
Find the best chunks for the question.

05
Augment
Combine question, chunks and instructions.

06
Generate
Answer from the chunks, with citations.

ON THE EXAM

The first three steps run when content changes. The last three run on every question.

Microsoft's RAG implementation stages

STAGE	WHAT YOU DO
Prepare your data	Organise and chunk documents into searchable content
Set up an index	Create an Azure AI Search index, the recommended store
Connect to Foundry	Create a project connection to the index
Build the application	Integrate retrieval with model calls via the Foundry SDK
Test and evaluate	Check retrieval quality, accuracy and citations

ON THE EXAM

Evaluation is a stage, not an afterthought. Retrieval settings are only proven by measuring them.

Why chunk size matters

- Embedding models have **input limits**, so whole long documents can't be embedded at once.
- A focused chunk **matches a specific question** better than a whole chapter does.
- **Too small:** a chunk loses the context that gives it meaning.
- **Too large:** relevance is diluted, and every retrieved chunk costs more tokens.
- There's no universal size. Test a few settings against real questions.

> ON THE EXAM
>
> Chunking is a trade-off between precision and context. Overlap and titles recover context without growing chunks.

Chunking techniques

TECHNIQUE	HOW IT WORKS	NOTES
Fixed size with overlap	Equal-sized chunks sharing a little text	Start with about 10% overlap
Variable by content	Split at sentences, lines or Markdown headings	Keeps ideas together
Structure-aware	Split by document layout: headings, tables	Content Understanding skill for new work
Add the title	Prefix each chunk with its document title	Stops mid-document chunks losing context

ON THE EXAM

Overlap carries sentences across boundaries. Titles tell a chunk which document it came from.

The Text Split skill

- **textSplitMode:** pages, the default, or sentences.
- **maximumPageLength:** chunk size in characters. The default is **5,000**.
- **pageOverlapLength:** overlap, which must be **less than half** the page length.
- **maximumPagesToTake:** how many chunks per document. The default, 0, means all.
- The splitter avoids cutting sentences, so chunk sizes vary slightly.

> ON THE EXAM
>
> Characters are not tokens. A 2,000-character chunk is roughly 500 tokens of English, but it varies.

Embedding chunks and questions

- **text-embedding-3-small** returns 1,536 dimensions by default; **-3-large** returns 3,072.
- The **dimensions** parameter shortens vectors, trading a little accuracy for storage and speed.
- Documents and questions must use the **same model and dimensions**.
- Call embeddings through the **/openai/v1** endpoint, as module 01 showed.
- Integrated vectorization in Azure AI Search can embed at indexing and at query time.

> ON THE EXAM
>
> Embedding happens twice: once per chunk at indexing, and once per question at query time.

Vectors from different models live in **different spaces**. Change the embedding model, and you **re-embed everything**.

Retrieval modes, in brief

MODE	FINDS	WEAKNESS
Keyword	Exact terms, codes, names	Misses different wording
Vector	Similar meaning	Can miss exact codes and names
Hybrid	Both, merged into one ranking	More work per query
Hybrid plus semantic ranker	Best results reordered by relevance	Extra cost and latency

ON THE EXAM

Hybrid with semantic ranking is the usual strong default. Module 31 covers how each mode scores results.

A classic RAG call

```
results = search.search(
    search_text=question,
    vector_queries=[VectorizableTextQuery(text=question,
        k_nearest_neighbors=5, fields="content_vector")],
    query_type="semantic", semantic_configuration_name="default", top=5)

sources = "\n".join(f"[{r['title']}] {r['content']}" for r in results)

answer = client.responses.create(model="chat-prod",
    instructions="Answer only from the sources. Cite titles in brackets. "
                 "If the sources don't contain the answer, say 'not found'.",
    input=f"Sources:\n{sources}\n\nQuestion: {question}")
```

ON THE EXAM

Retrieve with hybrid plus semantic ranking, then augment with titled sources and grounding instructions.

Building the grounded prompt

- Tell the model to **answer only from the supplied sources**.
- Ask for **citations**, using titles or IDs stored in the index.
- **Give an out:** 'not found' when the sources don't contain the answer.
- **Treat retrieved text as untrusted.** A document can contain injected instructions.
- **Stay within budget:** retrieve fewer, better chunks rather than many weak ones.

> ON THE EXAM
>
> Microsoft's fix for 'hallucination despite grounding' is citations plus clear instructions to stick to the sources.

Classic or agentic retrieval?

CLASSIC RAG

One query, fast

- A single search per question
- Lowest latency
- Suits simple, latency-critical questions

AGENTIC RETRIEVAL

Planned subqueries

- A model plans focused subqueries
- Uses conversation history for context
- Returns citations and metadata
- Adds latency

ON THE EXAM

Microsoft: start new RAG work with agentic retrieval; keep classic for simple, latency-critical queries.

Keeping RAG from leaking

- **Control access at retrieval time.** Azure AI Search supports document-level security filters.
- Filter results by the **signed-in user's identity or groups** before the model sees them.
- A system message saying 'don't reveal salaries' is **not access control**.
- Use **Microsoft Entra ID**, not keys, for the app's access to the search service.
- Foundry IQ knowledge bases can apply permission-aware retrieval for you.

> ON THE EXAM
>
> If a chunk reaches the prompt, assume it can reach the user. Trim at retrieval, not at generation.

RAG problems and Microsoft's fixes

PROBLEM	WHERE TO LOOK
Irrelevant passages retrieved	Chunking, embedding model, search mode
Invented answers despite grounding	Citations and clear instructions
Slow responses	Filtering, re-ranking, index design
Token budget exceeded	Fewer, filtered or summarised passages
Sensitive content in answers	Security filters at retrieval
Answers miss context across chunks	Overlap and titles on chunks

ON THE EXAM

Measure with the RAG evaluators from module 08 before and after each change.

> An HR assistant chunks a long handbook into 500-token pieces with no overlap. Conditions such as 'eligible after six months, unless a contractor' are often split across two chunks.
>
> Chunks from mid-document also don't say which policy they describe, so answers mix up policies.

What should the team change?

A Retrieve 50 chunks per question so both halves are usually present.

B Add about 10% overlap and prefix each chunk with its title.

C Switch to a larger chat model that reasons better over fragments.

D Store each whole handbook as a single chunk, so nothing gets split.

◆ PAUSE AND DECIDE

What should the team change?

A. Retrieve 50 chunks.

Costs many tokens, dilutes relevance, and still doesn't label mid-document chunks.

B. Overlap and titles.

Overlap keeps split conditions together; titles tell each chunk which policy it is.

C. Larger chat model.

A retrieval problem. The model can't reason about text it never receives.

D. Whole handbook per chunk.

Exceeds embedding input limits and makes every match vague.

THE SIGNAL

Two symptoms, two chunking fixes: split sentences need overlap, lost identity needs titles.

> A product search index was built with text-embedding-3-small. A developer changes the application's question embedding to text-embedding-3-large to improve quality.
>
> Vector results immediately become irrelevant. Keyword results are unaffected.

What should the team do?

A Request 1,536 dimensions from the large model to match the index.

B Turn on the semantic ranker to reorder the irrelevant vector results.

C Halve the chunk size and re-run the indexer on all product pages.

D Re-embed all documents with the large model, or revert the change.

◆ PAUSE AND DECIDE

What should the team do?

A Shorten large to 1,536.

Sizes match, but the vectors still come from a different model's space.

B Add the semantic ranker.

Reorders results, but can't repair a vector search comparing mismatched spaces.

C Smaller chunks.

Chunking isn't the fault. Re-indexing with the old model changes nothing.

D Same model on both sides.

Documents and questions must be embedded by the same model to be comparable.

THE SIGNAL

Vector results broke and keyword results didn't. Only the embedding side changed.

An internal assistant indexes every HR document, including salary spreadsheets, into one Azure AI Search index.

Testing shows any employee asking about pay can receive answers that quote other people's salaries.

What should the team do?

A Add a system message telling the model never to reveal salaries.

B Turn on the PII guardrail on output, set to annotate and block.

C Filter results by the user's groups with search security filters.

D Move the assistant to a separate model deployment with less quota.

◆ PAUSE AND DECIDE

What should the team do?

A System message rule.

A job description, not access control. It can be talked around.

B PII guardrail on output.

A salary figure isn't necessarily PII, and blocking late still retrieved it.

C Security filters at retrieval.

Unauthorised chunks never reach the prompt, so they can't reach the answer.

D Separate deployment.

Changes capacity, not who can see which documents.

THE TIEBREAKER

Trim at retrieval, not at generation. Microsoft's guidance: apply access control at retrieval time.

> A customer help assistant uses RAG over product manuals. When asked about products that aren't in the manuals, it gives confident, invented answers.
>
> Support staff can't tell which answers came from the manuals and which were made up.

What should the team change first?

A Tell the model: use only sources, cite titles, say 'not found'.

B Raise temperature so the model is less likely to repeat a pattern.

C Always retrieve many more chunks, so every question finds something.

D Fine-tune the chat model on the full set of product manuals instead.

◆ PAUSE AND DECIDE

What should the team change first?

A Sources only, citations, 'not found'.

Gives the model an out and makes every claim traceable to a manual.

B Raise temperature.

More variety means more invention, not less.

C Retrieve more chunks.

The answer isn't in the manuals. More irrelevant chunks invite more guessing.

D Fine-tune on manuals.

Fine-tuning shapes behaviour, not reliable facts, and removes citations.

THE SIGNAL

Invented answers plus 'can't tell what's sourced' is Microsoft's pairing: citations and clear grounding instructions.

RAG in an application, in five lines

- **Quality is decided before the model speaks:** chunk, embed, retrieve, instruct.
- **About 10% overlap, plus titles**, keeps context across chunks.
- **Same embedding model for documents and questions.** Change it, re-embed everything.
- **Trim access at retrieval.** Treat retrieved text as untrusted.
- **Sources only, citations, and 'not found'** fix invented answers.

> ON THE EXAM
>
> Next, module 14: workflows and multistep reasoning, including tool-augmented flows and self-critique.